

HW8 - Engineering Management

Student Hub Platform

Negative patterns (and how we avoid them)

1. Feature Creep (overloading the platform)

Problem:

Student platforms often try to include *everything* (chat, marketplace, notes, jobs, forums), which leads to a cluttered and confusing UI.

How we implement / avoid it:

- We define a clear MVP scope: e.g. courses, announcements, timetable, and basic student interaction.
- Features are grouped into modules (Dashboard, Study Tools, Community) instead of one overloaded page.
- We use a “progressive feature rollout” approach, new features only added after testing user need.

2. Poor Input Validation / Data inconsistency

Problem:

Without strict validation, users can enter incorrect data (wrong exam dates, broken links, duplicate posts).

How we implement it:

- Frontend validation (React form checks / required fields / formats).
- Backend validation (API-level enforcement so bad data cannot enter DB).
- Use of schema validation (e.g. Zod / Joi / Hibernate constraints depending on stack).
- Consistent error messages (so users understand what went wrong).

3. Lack of role separation (security + UX issue)

Problem:

If all users have the same permissions, students might access admin functions or sensitive data.

How we implement it:

- Implement **Role-Based Access Control (RBAC)**:
 - Student
 - Teaching assistant
 - Admin
- Protected routes in frontend + middleware checks in backend.
- Example: only admins can post official announcements.

4. Negative pattern: Tight coupling between frontend and backend

Problem:

If the frontend is too dependent on specific backend implementations (e.g., hardcoded endpoints, tightly bound data structures), even small backend changes can break the entire UI.

How we implement / avoid it:

- We define a stable API contract (clear request/response formats).
- Use API service layer in frontend (e.g. `apiService.js`) so components never call endpoints directly.
- Avoid exposing database structure to frontend.
- Version APIs if needed (`/api/v1/...`) to prevent breaking changes.

Result: backend and frontend can evolve independently without breaking the system.

Positive patterns (and how we implement them)

1. MVC / Clean separation of concerns

Why it matters:

Keeps the system maintainable and scalable.

How we implement it:

- **Frontend (React)** → View layer (UI components only)
- **Backend (Node.js / Spring Boot)** → Controller + business logic
- **Database layer** → Models / entities
- API acts as clean communication layer between frontend and backend

Result: easier debugging, easier team collaboration, easier scaling.

2. RESTful API design

Why it matters:

Makes the system predictable and scalable for future features (mobile app, integrations).

How we implement it:

- Clear endpoints:
 - GET /courses
 - POST /posts
 - DELETE /announcements/:id
- Stateless communication
- Proper HTTP status codes (200, 201, 400, 401, 404, 500)

This allows future expansion (mobile app or third-party integrations without redesigning backend).

3. Notification batching / Smart notifications

Why it matters:

Prevents users from being spammed and improves engagement.

How we implement it:

- Instead of instant notifications for everything → we batch updates
 - “You have 5 new course updates today”
- Users can set preferences:
 - instant / daily digest / off
- Use a notification queue system (e.g. Redis queue or scheduled jobs)

4. Positive pattern: Caching for performance optimization

Why it matters:

Student platforms often repeat the same requests (timetables, announcements, course lists), which can slow down performance if fetched every time.

How we implement it:

- Cache frequently accessed data:
 - Course lists
 - Announcements
 - User profile info
- Use:
 - Frontend caching (React Query / SWR)
 - Backend caching (Redis or in-memory cache)
- Set cache invalidation rules (e.g., refresh announcements every X minutes or on update)

Result: faster load times, reduced server load, and better user experience.